

Complete development journal. Every decision, every error, every fix. Paste this into a new chat to resume with full context.

Section 1 -- Project Identity

Field	Value
Project name	yt-agent2
One-line description	Converts any YouTube video into a publish-ready blog post using Groq Whisper + Llama 3.3 70B
Live Railway URL	(Set after successful deployment -- currently blocked by YouTube bot detection)
GitHub repository	(Push to GitHub repo -- URL TBD)
Local folder	C:\Users\user\Downloads\yt-agent2
Date started	June 23, 2026
Current status	Deployed on Railway. Build passes. App starts. Blocked: YouTube bot detection on Railway IPs. Fix documented in Section 8.

Section 2 -- Why This Project Exists

The Previous Project: yt-agent (6 agents)

The first version of this project (yt-agent) was built with a 6-agent architecture:

1. DownloadAgent -- downloaded YouTube audio
2. TranscriptAgent -- transcribed audio
3. ResearchAgent -- researched the transcript topic
4. OutlineAgent -- generated a blog post outline
5. WritingAgent -- wrote the blog post from the outline
6. ReviewAgent -- reviewed and polished the post

Why the Mentor Said to Simplify

The 6-agent architecture was over-engineered for the actual task. Each handoff between agents added latency, complexity, and failure points. The ReviewAgent rarely improved the output meaningfully. The OutlineAgent was a redundant intermediate step. The entire pipeline was hard to debug and slow to iterate on.

The Simplification Decision

Rebuild from scratch with exactly two agents and one orchestrator:

- TranscriptAgent -- owns everything related to getting a transcript (download + transcribe)
- BlogPostAgent -- owns everything related to producing a blog post (research + write, both internally)
- Orchestrator -- thin coordinator, no logic

What Was Kept vs Removed vs Rebuilt

Item	Decision	Reason
yt-dlp for download	Kept	Still best tool for YouTube audio extraction
Groq Whisper for transcription	Kept	Fast, accurate, free tier
Groq Llama for writing	Kept	Best free LLM for long-form content
ResearchAgent	Removed -- absorbed into BlogPostAgent	Research is not a standalone responsibility; it's prep work for writing
OutlineAgent	Removed -- absorbed into BlogPostAgent	Same reason
ReviewAgent	Removed	Not worth the latency for marginal quality gain
Streamlit UI	Rebuilt	Much cleaner with live log panel and better error display
Error handling	Rebuilt completely	New version classifies errors and acts intelligently

Section 3 -- Complete Tech Stack

Technology	Version	Purpose	Free / Paid
Python	3.12	Runtime	Free
yt-dlp	2026.6.9	Downloads YouTube audio streams	Free
ffmpeg	system (Nix)	Converts audio to MP3 for Whisper	Free
Groq API -- Whisper	whisper-large-v3-turbo	Audio transcription	Free tier

Groq API -- LLM	llama-3.3-70b-versatile	Research + blog post writing	Free tier
Streamlit	1.41.1	Web UI	Free
python-dotenv	1.2.2	Loads .env in development	Free
Railway	--	Cloud hosting	Free tier (500 hrs/month)
Nixpacks	--	Railway's build system	Free (built into Railway)

Groq Free Tier Rate Limits

Model	Requests/min	Tokens/min	Tokens/day
whisper-large-v3-turbo	20	--	~2 hours of audio
llama-3.3-70b-versatile	30	6,000	500,000

Why Each Was Chosen

- yt-dlp: Only reliable open-source tool that handles YouTube's constantly-changing download protection. Supports cookies, multiple player clients, and audio-only extraction.
- Groq: Fastest LLM inference available. whisper-large-v3-turbo is 10x faster than OpenAI Whisper at the same accuracy. llama-3.3-70b-versatile is a strong instruction-following model for long-form writing. Both on the free tier.
- Streamlit: Minimal boilerplate for a Python web app. st.status() provides a native progress panel. No frontend code needed.
- Railway: Best free-tier cloud platform for Python apps. Supports Nixpacks which lets us install system packages (ffmpeg) without a custom Dockerfile. Auto-deploys from GitHub.

Section 4 -- Architecture: Every Detail

The Three Components

```

YouTube URL
  |
  v
[TranscriptAgent] ??> plain-text transcript
  |
  v
[BlogPostAgent]    ??> Markdown blog post
  |
  v
[Orchestrator]     ??> { transcript: str, blog_post: str }
  |
  v
[app.py / Streamlit UI]

```

Exact Data Flow

Stage	Input	Output	Who calls it
TranscriptAgent.run()	YouTube URL (str)	plain-text transcript (str)	Orchestrator
BlogPostAgent.run()	transcript (str)	Markdown blog post (str)	Orchestrator
Orchestrator.run()	YouTube URL (str)	{"transcript": str, "blog_post": str}	app.py

Why the Orchestrator is Thin

The Orchestrator has zero business logic by design. It does not make decisions -- it only calls agents in order and passes data between them. This means:

1. Each agent can be tested independently (python agents/transcript_agent.py)
2. The pipeline sequence can be changed without touching agent code
3. Adding a new agent only requires one line in Orchestrator, not a refactor

Why Research Lives Inside BlogPostAgent (Not a Separate Agent)

Research is not a standalone capability -- it is preparation for writing. The research notes are never useful on their own; they only exist to make the writing better. Separating them into a ResearchAgent would mean:

- An extra agent with its own run() signature to maintain
- The writing agent would need to accept two inputs instead of one
- Research failure would crash the pipeline before writing even starts

By keeping research inside BlogPostAgent._research(), the fallback is natural: if research fails, _write() simply uses the raw transcript instead. The pipeline never crashes due to a research failure.

The Two LLM Calls in BlogPostAgent -- Exact Parameters

LLM Call 1: _research()

```
model      = "llama-3.3-70b-versatile"
temperature = 0.4          # Lower = more focused, analytical
max_tokens = 1500
transcript = _truncate(transcript, 6000) # 80% front, 20% back
```

Output: Structured research notes covering:

1. CORE TOPIC
2. TARGET AUDIENCE
3. KEY INSIGHTS (4-6)
4. HOOK ANGLE
5. SECTION STRUCTURE (3-5 headings)
6. HUMAN ELEMENT
7. CLOSING DIRECTION

LLM Call 2: _write()

```

model      = "llama-3.3-70b-versatile"
temperature = 0.7          # Higher = more creative writing
max_tokens = 2000
transcript = _truncate(transcript, 3000)  # used as supporting detail

```

Input: research_notes (or empty string if research failed) + truncated transcript

Output: 600-900 word Markdown blog post, starting with # Title, using ## Section headings, no bullet dumps.

If research_notes == "" (fallback path): transcript window expands to 4000 chars and notes section is removed from the prompt.

Error Classification in TranscriptAgent

The _classify() function inspects the lowercase error message string and returns one of seven error type strings:

```

def _classify(error_msg: str) -> str:
    msg = error_msg.lower()
    if "sign in" in msg or "confirm you're not a bot" in msg:
        return "bot_detected"
    if "requested format" in msg or "no video formats" in msg:
        return "format_error"
    if "429" in msg or "rate limit" in msg or "too many requests" in msg:
        return "rate_limit"
    if any(s in msg for s in ["video unavailable", "private video",
                             "this video is not available",
                             "video has been removed"]):
        return "video_unavailable"
    if "ffmpeg" in msg or "ffprobe" in msg:
        return "ffmpeg_error"
    if any(s in msg for s in ["timed out", "connection", "network"]):
        return "network_error"
    return "unknown"

```

Error Type Behaviour Table

Error Type	Category	Behaviour	User-Facing Message
bot_detected	UNRECOVERABLE	Stop immediately	"YouTube blocked this as a bot. Set YOUTUBE_COOKIES..."
video_unavailable	UNRECOVERABLE	Stop immediately	"This video is private, deleted, or region-restricted."
ffmpeg_error	UNRECOVERABLE	Stop immediately	"Audio processing failed -- ffmpeg is missing or broken."

rate_limit	BACKOFF_RETRY	Retry x3 with backoff	"Rate limited. Try again in a few minutes."
network_error	BACKOFF_RETRY	Retry x3 with backoff	"Network error. Check connection and try again."
format_error	neither	Retry once, then fail	"No compatible audio format found for this video."
unknown	neither	Retry x3	"An unexpected error occurred."

```
UNRECOVERABLE = {"bot_detected", "video_unavailable", "ffmpeg_error"}
BACKOFF_RETRY  = {"rate_limit", "network_error"}
```

Retry Logic

```
MAX_RETRIES = 3
Backoff formula: wait = 2 ** attempt (attempt 1 = 2s, attempt 2 = 4s)
```

Both `_download_with_retry()` and `_transcribe_with_retry()` use the same pattern:

1. Try the operation
2. On exception: classify the error
3. If UNRECOVERABLE: raise immediately with user-friendly message
4. If BACKOFF_RETRY and not last attempt: sleep and continue
5. If last attempt: raise with user-friendly message

Cookie Support

`_get_cookies_path()` checks the `YOUTUBE_COOKIES` environment variable:

1. If empty: returns None (no cookies used)
2. If value is an existing file path: returns that path directly
3. If value starts with `#` or `.` (cookie file content): writes content to a `NamedTemporaryFile` and returns the temp path

The cookiefile path is added to `yd_opts["cookiefile"]` only if non-None.

yt-dlp Player Clients

```
"extractor_args": {
    "youtube": {
        "player_client": ["android", "ios", "tv_embedded", "web"],
    }
}
```

yt-dlp tries these clients in order:

- android and ios: Mobile app clients -- do not require a JavaScript runtime to extract the video manifest. Most reliable for bypassing bot detection without cookies.
- tv_embedded: The YouTube TV embedded player -- less restricted than the web player.

web. Standard web browser client -- most likely to be bot-detected but included as fallback. Why androidios work without JS runtime: YouTube's web player extracts video URLs using JavaScript (the "nsig" cipher). Mobile app clients return video URLs pre-decoded in a different format that does not require JS execution.

js_runtimes Fix

The correct format for js_runtimes in yt-dlp is a dict, not a string:

```
# CORRECT
js_runtimes = {"node": {"path": "/path/to/node"}}

# WRONG -- causes TypeError crash
js_runtimes = "node"
```

This was a bug we hit during development (see Section 6).

Transcript Truncation -- _truncate()

```
def _truncate(self, text: str, max_chars: int) -> str:
    if len(text) <= max_chars:
        return text
    front = int(max_chars * 0.8) # 80% from start
    back = max_chars - front # 20% from end
    return text[:front] + "\n\n[...middle omitted...]\n\n" + text[-back:]
```

Rationale: The beginning of a video typically contains the most important context (introduction, thesis, main points). The end often contains conclusion or calls to action. The middle often has examples and elaboration that can be safely compressed.

Section 5 -- Complete File Reference

`agents/transcript\_agent.py`

Purpose: Agent 1. Downloads YouTube audio and transcribes it to plain text.

Key classes/functions:

- `_classify(error_msg)` -- module-level function, returns error type string
- `UNRECOVERABLE`, `BACKOFF_RETRY`, `USER_MESSAGES` -- module-level constants
- `TranscriptAgent` -- main class
 - `run(youtube_url)` -- public entry point, returns transcript string
 - `_download_with_retry(url, output_dir)` -- retry wrapper for download
 - `_download_audio(url, output_dir)` -- core yt-dlp download logic
 - `_find_audio_file(directory)` -- scans output dir for downloaded audio
 - `_get_cookies_path()` -- reads YOUTUBE_COOKIES env var, returns file path or None
 - `_transcribe_with_retry(audio_path)` -- retry wrapper for Whisper call
 - `_validate_url(url)` -- basic URL sanity check

Key implementation details:

audio[ext=m4a]/bestaudio[ext=webm]/bestaudio/best"

- Codec: MP3, 64 kbps (keeps files small for Whisper's 25 MB limit)

- Whisper call: model="whisper-large-v3-turbo", response_format="text", temperature=0.0
- MAX_FILE_SIZE_MB = 24 (1 MB margin below Whisper's 25 MB limit)
- MAX_RETRIES = 3
- Uses tempfile.TemporaryDirectory() -- audio file is deleted after transcription
- import re is imported but not currently used (can be removed in future cleanup)

Known issues / improvement notes:

- import re is unused -- minor cleanup opportunity
- Cookie temp file is written with delete=False but never explicitly cleaned up -- low priority since it's in the system temp dir
- The unknown error type retries 3 times even for truly unrecoverable errors not yet in the classifier -- consider adding more patterns

`agents/blog\_post\_agent.py`

Purpose: Agent 2. Makes two LLM calls to produce a structured blog post from a transcript.

Key classes/functions:

- BlogPostAgent -- main class
 - run(transcript) -- public entry point, returns Markdown blog post
 - _research(transcript) -- LLM call 1, returns research notes string
 - _write(transcript, research_notes) -- LLM call 2, returns blog post
 - _call_llm(system_prompt, user_prompt, max_tokens, temperature) -- shared retry-wrapped LLM caller
 - _truncate(text, max_chars) -- 80/20 truncation helper
 - _validate_transcript(transcript) -- min 100 chars check

Key implementation details:

- MODEL = "llama-3.3-70b-versatile" -- class constant, easy to change
- MAX_RETRIES = 3
- Research failure is caught in run() and falls back to research_notes = ""
- 401 errors in _call_llm() raise immediately (bad API key -- no point retrying)
- Research prompt uses 6000 char window; write prompt uses 3000 chars (with notes) or 4000 chars (without notes)

Known issues / improvement notes:

- max_tokens=2000 sometimes produces slightly truncated posts for complex topics -- could increase to 2500
- Research notes are not shown to the user; could add an expander in the UI for debugging
- No language parameter in LLM calls -- already language-agnostic for writing
- Temperature 0.7 for writing occasionally produces slightly inconsistent tone -- could experiment with 0.65

`orchestrator.py`

Purpose: Thin coordinator. Instantiates agents, calls them in sequence.

Key classes/functions:

- Orchestrator


```
def __init__(self):  
    # creates self.transcript_agent and self.blog_post_agent  
    self.transcript_agent = TranscriptAgent()  
    self.blog_post_agent = BlogPostAgent()  
    # run(youtube_url) -- calls agents in sequence, returns {"transcript": ..., "blog_post": ...}
```

Key implementation details:

- No error handling -- all exceptions propagate up to app.py
- Agent instances are created once in `__init__()` and reused (Groq client is stateless so this is safe)
- In app.py, the agent `.run()` methods are monkey-patched to inject Streamlit progress messages -- this works because `orch.transcript_agent.run` is an instance attribute after patching

Known issues / improvement notes:

- None. This file is intentionally minimal and correct as-is.

`app.py`

Purpose: Streamlit web UI. Entry point for Railway.

Key functions:

- `append_log(msg)` -- appends a line to `log_lines` and refreshes `log_area` (a `st.empty()` placeholder)
- `t_run_patched(youtube_url)` -- wraps `transcript_agent.run()` with log messages
- `b_run_patched(transcript)` -- wraps `blog_post_agent.run()` with log messages

Key implementation details:

- `os.environ.get("GROQ_API_KEY")` check runs on every page load -- shows warning banner immediately if key is missing
- `st.status()` provides the collapsible progress panel
- `log_area = st.empty()` is created inside `st.status()` -- the `append_log()` closure captures it via the outer scope
- Agent methods are monkey-patched after `Orchestrator()` init: `orch.transcript_agent.run = t_run_patched`
- Output tabs: "Blog Post" (rendered Markdown + download button) and "Raw Transcript" (text area with word/char count)
- `st.stop()` is called after `st.error()` to prevent the output section from rendering on error

Known issues / improvement notes:

- The `log_area.code()` call re-renders the entire log on every new line -- could be expensive for very large logs; acceptable for current scale
- No session state -- if user hits back and resubmits, no history. Future improvement: `st.session_state`.

`requirements.txt`

Purpose: Exact pinned Python dependencies for reproducible installs.

Contents:

```
yt-dlp==2026.6.9  
groq==1.4.0  
python-dotenv==1.2.2  
streamlit==1.41.1
```

Important notes:

- These are exact pins (`==`), not ranges. This ensures Railway always installs the same versions that were

- Never use pip freeze to update this file -- on Windows, pip freeze captures Windows-only packages like pywin32, pywinpty, colorama that do not exist on Linux and will crash Railway's build. Always update manually.
 - ffmpeg is a system dependency -- it is not in requirements.txt. It is installed via nixpacks.toml.
-

`railway.toml`

Purpose: Railway deployment configuration -- build system, start command, health check, restart policy.

Current contents:

```
[build]
builder = "nixpacks"

[deploy]
startCommand = "PYTHONPATH=/app streamlit run app.py --server.port $PORT --server.address 0.0.0.0 --server.headless true"
healthcheckPath = "/_stcore/health"
healthcheckTimeout = 300
restartPolicyType = "ON_FAILURE"
restartPolicyMaxRetries = 3
```

Line-by-line explanation:

- builder = "nixpacks" -- use Nixpacks (reads nixpacks.toml for system packages)
- startCommand -- the command Railway runs to start the app
 - PYTHONPATH=/app -- tells Python to look for modules in /app (Railway's working directory). Without this, from agents.transcript_agent import TranscriptAgent fails with ModuleNotFoundError.
 - streamlit run app.py -- starts the Streamlit server
 - --server.port \$PORT -- binds to the port Railway assigns at runtime
 - --server.address 0.0.0.0 -- listens on all interfaces (required for Railway's reverse proxy)
 - --server.headless true -- suppresses Streamlit's browser-open attempt and email prompt (required in headless server environment)
- healthcheckPath = "/_stcore/health" -- Railway polls this endpoint to verify the app is running. This is Streamlit's built-in health endpoint. Using / doesn't work reliably because Streamlit's root redirects.
- healthcheckTimeout = 300 -- Railway waits up to 5 minutes for the health check to pass. Needed because first-time builds are slow.
- restartPolicyType = "ON_FAILURE" -- Railway restarts the service if it crashes.
- restartPolicyMaxRetries = 3 -- Maximum 3 automatic restarts before Railway stops trying.

NOTE: In an earlier version we had buildCommand = "pip install -r requirements.txt" and used lowercase restartPolicyType = "on-failure". Railway rejected the lowercase value. The correct format is "ON_FAILURE". The buildCommand line was removed because Nixpacks auto-detects requirements.txt.

`nixpacks.toml`

Purpose: Tells Nixpacks which system packages to install during the Railway build.

Current contents:

```
[phases.setup]
nixPkgs = ["ffmpeg", "python312"]
```

Why these two packages:

- ffmpeg -- required by yt-dlp's FFmpegExtractAudio post-processor. Without it, yt-dlp downloads the audio stream but cannot convert it to MP3, and the build fails silently with "ffmpeg not found".
- python312 -- pins Python to exactly 3.12. Without this, Nixpacks may pick a different Python version, causing environment mismatches between the Nix-installed packages and the pip-installed packages.

`.env.example`

Purpose: Template showing required environment variables. Committed to version control.

Contents:

```
# Copy this file to .env and fill in your key.
# Get your free key at: https://console.groq.com

GROQ_API_KEY=your_groq_api_key_here
```

Notes: The actual .env file is gitignored. On Railway, set variables in the Railway dashboard under Variables, not via a .env file.

`.gitignore`

Status: .gitignore was found in the parent directory (C:\Users\user\Downloads\l) rather than inside the project folder. Recommend creating one inside yt-agent2/.

Minimum contents it should have:

```
.env
__pycache__/
*.pyc
*.pyo
venv/
.venv/
*.egg-info/
dist/
build/
```

`.README.md`

Purpose: Public-facing project documentation for GitHub.

Sections: Title + badges, Overview, How It Works (pipeline diagram), Features, Tech Stack table, Project Structure, Local Setup (with ffmpeg install commands), Railway Deployment, Environment Variables table, YouTube Bot Detection (fix guide), How the Agents Work (error table + LLM call table), Limitations table, Contributing, License, Author.

`CLAUDE.md`

Status: Does not exist. Not needed for this project.

Section 6 -- Complete Build Journey

Step 1: Project Created

- Created yt-agent2/ folder with agents/ subdirectory
 - Created requirements.txt with initial package list (using >= ranges at first)
 - Created .env and .env.example
 - Added GROQ_API_KEY to .env from Groq console
-

Step 2: TranscriptAgent -- First Version

- Built agents/transcript_agent.py with basic yt-dlp download + Groq Whisper transcription
 - Initial format string: "format": "worstaudio/bestaudio" -- this worked locally but later caused issues on Railway with certain videos
 - No error classification -- just a plain try/except RuntimeError
 - Tested standalone with python agents/transcript_agent.py using <https://www.youtube.com/watch?v=arj7oStGLkU> (60-second TED-Ed clip)
 - Result: PASSED locally
-

Step 3: BlogPostAgent -- First Version

- Built agents/blog_post_agent.py with two separate methods (_research, _write), each making their own LLM call with duplicated try/except
 - Tested standalone with a hardcoded transcript sample (procrastination TED-Ed content)
 - Result: PASSED. Blog post quality was good.
-

Step 4: Orchestrator Built and Tested

- Built orchestrator.py -- 40 lines, pure coordination
 - Full pipeline test with real YouTube URL
 - Result: PASSED. Full pipeline URL -> transcript -> blog post working end-to-end locally.
-

Step 5: Streamlit UI -- First Version

- Built app.py with basic st.text_input, st.button, st.status() panel
 - Initial version used simple st.write() calls for progress -- no live log panel
-

Step 6: Railway Deployment -- First Attempt

Files created: Procfile, railway.toml, nixpacks.toml

Error 1: restartPolicyType rejected

```
railway.toml parse error: unknown field `restartPolicyType`
```

- Root cause: Used restartPolicyType = "on-failure" (lowercase)
- Railway's schema requires "ON_FAILURE" (uppercase) -- but even then, this field caused issues in some Railway versions
- Fix: Removed restartPolicyType from the file entirely in the early version. Later re-added with correct uppercase format once Railway schema was confirmed.

Error 2: ffmpeg not found at runtime

```
ERROR: ffmpeg not found. Please install ffmpeg.
```

- Root cause: nixpacks.toml only had nixPkgs = ["ffmpeg"] without pinning Python version. Nixpacks picked a different Python environment than the one with ffmpeg on PATH.
- Fix: Added python312 to nixPkgs: nixPkgs = ["ffmpeg", "python312"]

Error 3: ModuleNotFoundError: No module named 'agents'

```
ModuleNotFoundError: No module named 'agents'
```

- Root cause: Railway deploys the project to /app inside the container. Python's sys.path does not include /app by default in some Nixpacks environments, so from agents.transcript_agent import TranscriptAgent failed.
- Fix: Prepend PYTHONPATH=/app to the start command:

```
PYTHONPATH=/app streamlit run app.py ...
```

Step 7: requirements.txt Pinning Issue

- Early version of requirements.txt used >= ranges and comments
- At one point, pip freeze was considered for pinning -- this was not done because pip freeze on Windows captures Windows-only packages:
 - pywin32
 - pywinpty
 - colorama
 - wincertstore
- These packages do not exist on Linux and cause Railway's pip install to fail
- Fix: Requirements pinned manually to exact versions known to work

- Initial format "format": "worstaudio/bestaudio" works for most videos but fails for some YouTube videos that restructured their format offerings
- yt-dlp reports: "requested format not available" for certain videos
- Fix: Changed to more specific format chain:

```
"format": "bestaudio[ext=m4a]/bestaudio[ext=webm]/bestaudio/best"
```

This tries M4A first (most compatible with ffmpeg), then WebM, then any best audio, then full best stream.

Step 9: js_runtimes Dict Format Bug

- When adding Node.js runtime detection for yt-dlp, the initial attempt used:

```
js_runtimes = "node" # WRONG
```

- This caused a TypeError crash inside yt-dlp
- Fix: Correct format is a dict:

```
js_runtimes = {"node": {"path": "/path/to/node"}}
```

Step 10: Agent Rewrites -- Agentic Behaviour Added

All three files replaced with production versions featuring:

- TranscriptAgent: Error classification, retry with backoff, unrecoverable error detection, multiple player clients, cookie support, browser User-Agent header
 - BlogPostAgent: Shared _call_llm() with retry, non-fatal research fallback, intelligent transcript truncation, rate limit detection with 401 fast-fail
 - Orchestrator: Cleaned up logging labels
-

Step 11: app.py Rewrite -- Live Log Panel

- Replaced simple st.write() calls with append_log() + st.empty() live log pattern
 - Added GROQ_API_KEY check on page load (not just on submit)
 - Added word/char count to transcript tab
-

Step 12: Current Blocker -- YouTube Bot Detection

- Deployment is live on Railway. Build passes. App starts. Health check passes.
- When a YouTube URL is submitted, yt-dlp returns:

```
ERROR: [youtube]: Sign in to confirm you're not a bot.
```

- Root cause: Railway's datacenter IP range is flagged by YouTube's bot detection
 - Status: Not yet fixed. Fix approach documented in Section 8.
-
-

railway.toml -- Exact Current Contents

```
[build]
builder = "nixpacks"

[deploy]
startCommand = "PYTHONPATH=/app streamlit run app.py --server.port $PORT --server.address 0.0.0.0 --server.headless true"
healthcheckPath = "/_stcore/health"
healthcheckTimeout = 300
restartPolicyType = "ON_FAILURE"
restartPolicyMaxRetries = 3
```

nixpacks.toml -- Exact Current Contents

```
[phases.setup]
nixPkgs = ["ffmpeg", "python312"]
```

Environment Variables

Variable	Required	Default	Purpose
GROQ_API_KEY	YES	none	Groq API key for both Whisper and Llama. Get free at console.groq.com
YOUTUBE_COOKIES	NO	""	Cookie bypass for YouTube bot detection. File path or raw Netscape cookie file content

Start Command -- Every Flag Explained

```
PYTHONPATH=/app streamlit run app.py --server.port $PORT --server.address 0.0.0.0 --server.headless true
```

Part	Purpose
PYTHONPATH=/app	Adds Railway's working directory to Python's module search path. Required for from agents.transcript_agent import ... to resolve.
streamlit run app.py	Starts the Streamlit server using app.py as the entry point.
--server.port \$PORT	Uses Railway's dynamically assigned port. \$PORT is injected by Railway at runtime.

<code>--server.address 0.0.0.0</code>	Binds to all network interfaces. Without this, Streamlit only listens on localhost and Railway's reverse proxy cannot reach it.
<code>--server.headless true</code>	Prevents Streamlit from trying to open a browser and showing the email subscription prompt. Required in server environments.

Health Check

- Path: `/_stcore/health` -- Streamlit's built-in health endpoint, returns 200 OK once the server is ready
- Why not `/`: Streamlit's root URL redirects and may not return a stable 200 during startup
- Timeout: 300 seconds -- Railway waits up to 5 minutes for first health check to pass

Section 8 -- Known Issues and Status

Issue 1: YouTube Bot Detection (CURRENT BLOCKER)

Status: Active. Blocks all YouTube URL submissions on the Railway deployment.

Root cause: YouTube's server-side bot detection flags requests from:

1. Known datacenter IP ranges (Railway uses AWS/GCP infrastructure)
2. Requests with no browser cookies (yt-dlp headless requests have none by default)
3. High-frequency request patterns from a single IP

The error message from yt-dlp:

```
ERROR: [youtube] XXXXXXXXXXXX: Sign in to confirm you're not a bot.
      This helps protect our community. Learn more
```

Current mitigation in code: yt-dlp is configured to try android, ios, tv_embedded, web player clients and sends a real Chrome User-Agent header. This works sometimes but not reliably on Railway IPs.

Fix -- Cookie Bypass:

Step 1: Install the Chrome extension "Get cookies.txt LOCALLY"

- Chrome Web Store: search "Get cookies.txt LOCALLY"

Step 2: Export cookies

- Go to <https://www.youtube.com> while signed in to a Google account
- Click the extension icon -> Export -> save as cookies.txt
- The file is in Netscape HTTP Cookie File format -- looks like:

```
# Netscape HTTP Cookie File
.youtube.com TRUE / FALSE 1234567890 CONSENT YES+...
.youtube.com TRUE / TRUE 1234567890 __Secure-1PSID ...
```

Step 3: Set on Railway

- Railway Dashboard -> your project -> Variables tab

- Value: either the full path to cookies.txt (if you can place the file in the repo), or paste the entire file contents directly as the variable value

Step 4: The `_get_cookies_path()` method in `TranscriptAgent` handles both cases automatically.

Important: Cookies expire. If bot detection returns after working, re-export fresh cookies.

Issue 2: Groq Whisper 25 MB File Limit

Status: Handled. Not a blocker for typical videos.

Videos longer than approximately 30 minutes at 64 kbps MP3 will exceed 25 MB and fail. `TranscriptAgent` checks file size after download and raises a clear error. No fix needed -- this is a Groq API limit.

Issue 3: Groq Rate Limits on Free Tier

Status: Handled via retry logic. Not a blocker.

If the Groq API returns a 429 (rate limit), both `TranscriptAgent._transcribe_with_retry()` and `BlogPostAgent._call_llm()` will back off and retry automatically.

Section 9 -- What Was Tested and Results

TranscriptAgent Standalone Test

```
python agents/transcript_agent.py
```

- URL tested: <https://www.youtube.com/watch?v=arj7oStGlkU> (TED-Ed "Inside the mind of a master procrastinator" -- 60 seconds)
 - Result: PASSED locally. Transcript returned ~800 characters of accurate English text.
 - On Railway: FAILS with bot detection error.
-

BlogPostAgent Standalone Test

```
python agents/blog_post_agent.py
```

- Input: Hardcoded procrastination transcript (~700 chars)
 - Result: PASSED. Blog post generated, ~700-850 words, clean Markdown with title and section headings.
-

Orchestrator Full Pipeline Test

- URL tested: <https://www.youtube.com/watch?v=arj7oStGLkU>
- Result: PASSED locally. Both transcript and blog post returned successfully.

Railway Deployment

Stage	Status	Notes
Git push	PASSED	Code pushed to Railway via GitHub connection
Nixpacks build	PASSED	Python 3.12 + ffmpeg installed, pip packages installed
App startup	PASSED	Streamlit starts, health check passes
YouTube URL submission	BLOCKED	Bot detection error from YouTube

Section 10 -- Things To Never Do Again

Rule	Why
Never use pip freeze to generate requirements.txt on Windows	Captures Windows-only packages (pywin32, pywinpty, etc.) that fail on Railway's Linux container
Never use js_runtimes = "node" (string format)	yt-dlp expects a dict: {"node": {"path": "..."/>
Never use "format": "worstaudio/bestaudio" as the only yt-dlp format	Fails for some video types; use "bestaudio[ext=m4a]/bestaudio[ext=webm]/bestaudio/best"
Never use lowercase restartPolicyType = "on-failure" in railway.toml	Railway schema requires uppercase "ON_FAILURE"
Always set PYTHONPATH=/app in the Railway start command	Without it, from agents.x import Y fails with ModuleNotFoundError
Always use /_stcore/health for Streamlit healthcheck	/ does not return a reliable 200 during startup
Never commit .env	Contains real API keys
Never use buildCommand in railway.toml when Nixpacks can auto-detect requirements.txt	Redundant, and can conflict with Nixpacks' own install phase
Always pin exact versions in requirements.txt	Ranges (>=) can install versions that break the app on Railway

Section 11 -- What Comes Next (Roadmap)

1. Cookie bypass for YouTube bot detection -- export cookies from Chrome, set YOUTUBE_COOKIES in Railway Variables (see Section 8 for exact steps)

Short-Term (Quality Improvements)

2. Prompt tuning -- increase max_tokens to 2500 in _write(), add explicit word count instruction, test with 5+ video types
3. Multi-language support -- remove language="en" from Whisper call in _transcribe_with_retry(), add language auto-detection
4. Video length guard -- before downloading, check video duration via yt-dlp metadata and warn user if > 20 minutes

Medium-Term (UX Improvements)

5. Streaming output -- use st.write_stream() with Groq's streaming API to show blog post appearing word-by-word
6. Session history -- store past generations in st.session_state so users can flip between previous results
7. Custom tone selector -- st.selectbox with options: Professional, Casual, Technical, Storytelling -- passed into the writing system prompt

Long-Term (Feature Additions)

8. SEO metadata output -- add a third output tab: meta title, meta description, slug, tags
9. Direct publish -- Ghost API, Substack API, or Medium API integration
10. Batch processing -- accept multiple URLs, process sequentially, download all as a zip

Section 12 -- How To Continue In A New Chat

Exact Prompt To Paste

```
I am continuing development of yt-agent2 -- a Python/Streamlit app that
converts YouTube videos into blog posts using Groq Whisper + Llama 3.3 70B.

Full context is in PROJECT_JOURNAL.md at the project root. Please read it first.

Current state:
- All code is complete and working locally
- Deployed on Railway -- build passes, app starts
- BLOCKED: YouTube bot detection on Railway IPs (Section 8 of journal has the fix)

Local path: C:\Users\user\Downloads\yt-agent2

The immediate task is: [describe what you want to do next]
```

Most Important Context To Carry Forward

1. The bot detection fix -- cookies need to be exported from Chrome and set as YOUTUBE_COOKIES in Railway Variables
2. Never use pip freeze on Windows -- always pin requirements.txt manually

4. js_runtimes must be a dict -- not a string

5. Research failure is non-fatal -- by design, fallback to transcript-only writing

Current Working State of Every Component

Component	Local	Railway
TranscriptAgent -- download	Working	Blocked (bot detection)
TranscriptAgent -- transcribe	Working	Not reached (blocked above)
BlogPostAgent -- research	Working	Not reached
BlogPostAgent -- write	Working	Not reached
Orchestrator	Working	Not reached
app.py / Streamlit UI	Working	Starts, health check passes
Railway build	N/A	Passing
Railway deploy	N/A	Passing
End-to-end pipeline	Working	Blocked

Last updated: June 23, 2026